INFO SESSION

QuantumConnect

Particle Collision Challenge

May the best heuristic win.



Cristian Schuszter, PhD

Tech Lead / Software Architect · CERN

© Cristian Schuszter & CERN · QuantumConnect Info Session

Outline

1. The game
2. The arena
3. Build your bot
4. Test & submit
5. House rules
6. Compete & Q&A

The challenge

Write a bot using Java. Upload a JAR. Let it fight.

Friday @ noon

deadline

8×12

quantum field

6

connected quarks to win

2

prizes

No ML required (and discouraged). No external services. One Java class, one JAR.

If you can write a loop and compare numbers, you can compete.

Register a team → write your bot → upload the JAR → every bot plays every bot

© Cristian Schuszter & CERN · QuantumConnect Info Session

The game: Connect 4, scaled up

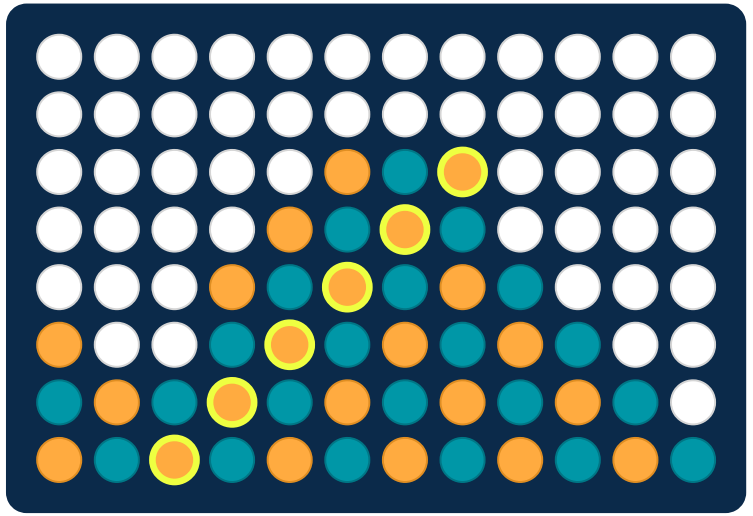
8 × 12 rows × columns. Gravity decides where your quark lands.

6 in a row horizontal, vertical or diagonal — six connected quarks win.

2 bots alternating turns. The arena alternates who starts.

1 shared field both bots see the same board. No hidden information.

This is not classic Connect 4: the board is wider and the win line is longer.



quantum field

first mover second mover

© Cristian Schuszter & CERN · QuantumConnect Info Session

A turn, step by step

100 ms / call

1 Call accelerateQuark

You get the field and 100 ms.

2 Return a column

0 … 11, and it must not be full.

3 The quark falls

Lowest free row in that column.

4 Win check

Six in a row? The match ends.

5 Call useSpecialMove

Highly recommended; 100 ms.

6 Switch player

Repeat until the field is full.

Both methods are called every turn. The field is read-only: getField() gives you a copy.

Exceed 100 ms or return a full column: the turn is skipped, not a crash.

© Cristian Schuszter & CERN · QuantumConnect Info Session

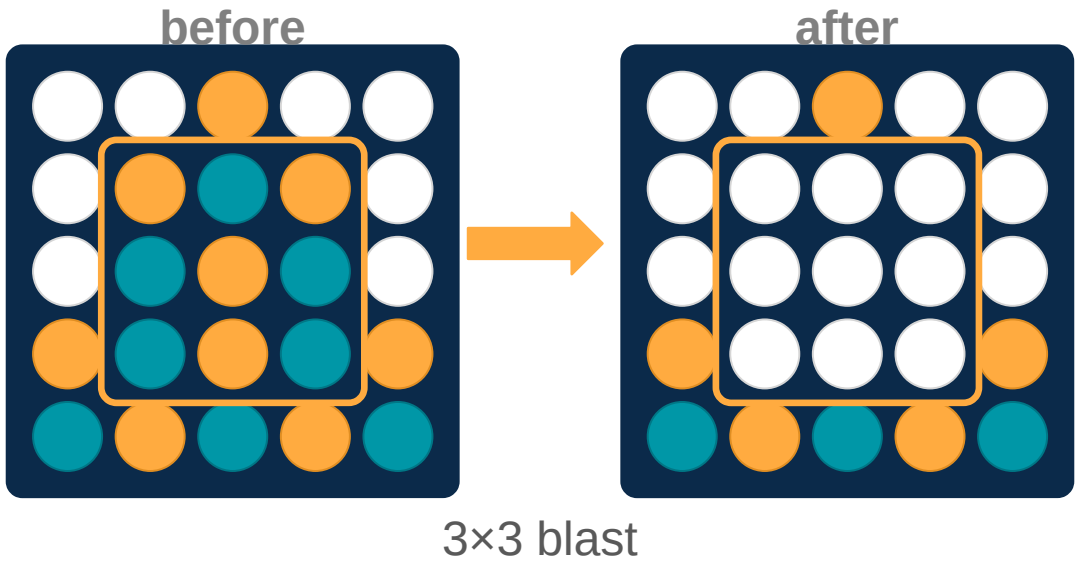
The twist: the special move

- **Return true**
  from useSpecialMove. The arena asks you after every move.
- **Your disc still lands**

  It uses the same column you returned.
- **3×3 annihilation**

  Every disc around it is destroyed — including yours.
- **Gravity intervenes**

  Discs above fall down to fill the holes.
- **Cooldown: 10 moves**
  Once every 10 of your own moves. Count before you detonate.



3×3 blast

cooldown: 10 own moves

A special move can erase an opponent's 5-in-a-row. It can also erase your own.

© Cristian Schuszter & CERN · QuantumConnect Info Session

How a match ends

WIN

Six connected quarks in any direction. The experiment collapses.



six in a row

DRAW

The field is full and nobody connected six. The result is uncertain.



field full

LOSS

Forbidden package, reflection, or breaking the sandbox. The bot automatically loses.



match lost

Per move: 100 ms. Match limit: 30 s. Every move is logged.

Exceed 100 ms or return a full column? The turn is skipped. A crash or exception can end the match.

© Cristian Schuszter & CERN · QuantumConnect Info Session

The arena: round-robin, not single elimination

Register a team

2-4 people, one account, activate by email.

One bot per team

Re-upload replaces it and deletes its old matches.

7 games per pair

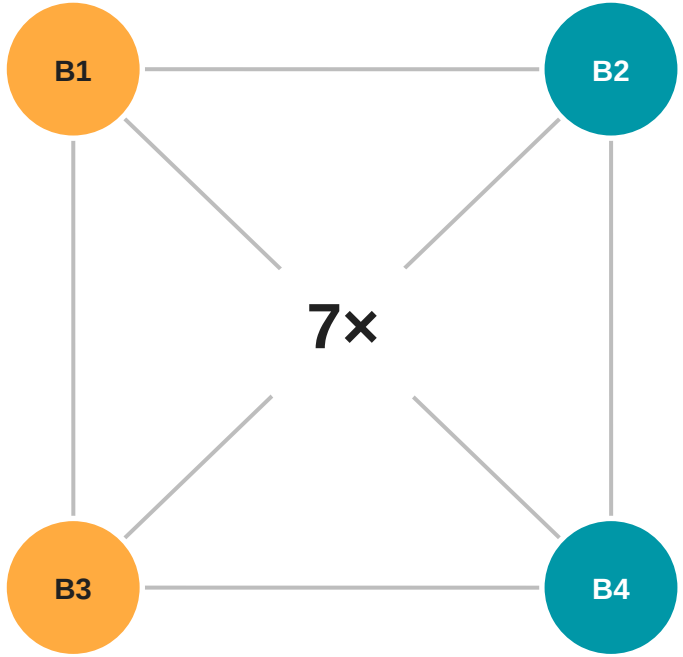
Every bot plays every other bot, alternating who starts.

Ranking

Wins first, then draws. Losses are shown, not hidden.

Hidden until reveal

The organisers can hide the leaderboard and block submissions while you code.



games per pair

After the deadline the arena reschedules and plays every match automatically. We'll pick the nicest ones for the final demo.

© Cristian Schuszter & CERN · QuantumConnect Info Session

Your 5 to 9 :)

Submissions only open in the evenings after lectures. Friday on the whole day.
Leaderboard may be frozen or hidden at times.

Monday - Wednesday

Initial development

Wednesday

Office hours at 5 PM.

Friday

Leaderboard freezes early in the morning.

Outline

1. The game
2. The arena
3. **Build your bot**
4. Test & submit
5. House rules
6. Compete & Q&A

Anatomy of a bot

One class
implements
ch.cern.quantumconnect.core.Algorithm.

Public no-arg constructor

the arena creates it with newInstance().

Two methods
accelerateQuark is required; **useSpecialMove** is highly recommended - otherwise the special move is never used.

Packaging

Create a JAR and upload it

```
public class MyBot implements Algorithm {

  public MyBot() { }  // public, no arguments

  @Override
  public int accelerateQuark(QuantumField field) {
    var cols = field.getNumberOfColumns();
    for (int col = 0; col < cols; col++) {
      if (!field.isColumnFull(col)) {
        return col;         // a legal column: 0..11
      }
    }
    return 0;               // board full: match is over
  }

  @Override
  public boolean useSpecialMove(QuantumField field) {
    return false;           // or true to detonate
  }
}
```

The starter project already has some examples. Use them.

© Cristian Schuszter & CERN · QuantumConnect Info Session

Legal moves

1. **Legal**
   Return any column that is not full.
2. **Win**
   Find the column that completes your six.
3. **Block**
   Find the column that completes theirs.
4. **Build**
   Prefer the center. Grow in two direction.
5. **Look ahead**
   Simulate the future?
6. **Detonate**
   Use the special move when it improves the board and your outcome.

```
var board = f.getField();
var me = myOwnership(f, board);

// 1. win now
for (int c = 0; c < f.getNumberOfColumns(); c++)
  if (!f.isColumnFull(c) && winsFor(board, c, me))
    return c;

// 2. block the opponent
var them = opponent(me);
for (int c = 0; c < f.getNumberOfColumns(); c++)
  if (!f.isColumnFull(c) && winsFor(board, c, them))
    return c;

// 3. build: centre-out
return centreOut(f);
```

© Cristian Schuszter & CERN · QuantumConnect Info Session

Test locally before you submit

- **Edit EmulatorRunner**
  new Experiment(new MyBot(), new RandomBot())
- **Run**
  ./gradlew run - a window shows the match.
- **Go headless & fast**
  new Experiment(a, b, false, false)
- **Run 100 games**
  Print a win rate; aim for >90% vs RandomBot.

Test the edge cases: full columns, a full board, and special moves.

```
$ unzip starter-bot-project.zip
$ cd qc-starter
# add MyBot.java, edit EmulatorRunner
$ ./gradlew run
```

```
void main() throws InterruptedException {
  var exp = new Experiment(new MyBot(),
                           new RandomBot());
  exp.runExperiment();
}
```

© Cristian Schuszter & CERN · QuantumConnect Info Session

Package & submit

1. **Set Bot-Class**

   build.gradle → the fully qualified class name.

2. **Build the JAR**

   ./gradlew build

3. **Find the artifact**

   build/libs/quantumconnect-starter-1.0-SNAPSHOT.jar

4. **Upload & verify**

   Bots page → name + JAR → Upload. Check that it appears.

```
jar {
  manifest {
    attributes(
      'Bot-Class': 'ch.qc.starter.MyBot'
    )
  }
}
```

Before you upload

- **Bot name: 5–20 characters, unique this season.**
- **JAR: .jar only, max 5 MB.**
- **One bot per team; re-upload replaces the old one.**
- **Upload early - the deadline is hard.**

© Cristian Schuszter & CERN · QuantumConnect Info Session

Outline

1. The game
2. The arena
3. Build your bot
4. Test & submit
5. **House rules**
6. Compete & Q&A

What you may use and not

ALLOWED

- Java standard library for computation: collections, arrays, math, streams.
- The engine jar in libs/: Algorithm, QuantumField, Move, …
- Your own classes and helper methods.
- The starter project's Gradle build (Java 25).
- Your IDE and the Java docs.

FORBIDDEN

- Network & files: java.net, java.io.
- Reflection is forbidden: java.lang.reflect, sun.*, com.sun.*.
- Security & crypto: java.security, javax.crypto.
- Threads & concurrency: java.lang.Thread*, java.util.concurrent.*.
- Processes, runtime and system access.
- Breaking the sandbox or attacking the arena.

Non-basic Java libraries and reflection are forbidden - forbidden access means the match is lost.

The arena loads your JAR in a restricted class loader. 100 ms per move; 30 s match limit.

© Cristian Schuszter & CERN · QuantumConnect Info Session

Performance & pitfalls

DO

Copy getField() once per turn; simulate on your copy.

Use arrays and primitive loops.

Handle full columns and a full board.

Seed Random for repeatable tests.

Keep a working upload at all times.

DON'T

Don't print in the hot path - stdout is captured.

Don't assume you are FIRST or SECOND; use belongsToMe().

Don't recurse without a depth limit.

Don't cache state across games; new instance, new match.

Don't throw exceptions as control flow.

Aim for <10 ms per move. The arena machine may be slower than your laptop.

100 ms is millions of Java operations - enough for a deep search if you keep it tight.

© Cristian Schuszter & CERN · QuantumConnect Info Session

Thank you.

Now go build something that collides.

Rules & API	/rules
Starter project & upload	/bots
Email	cristian.schuszter@cern.ch

45 minutes · 8×12 · 6 in a row · two winning teams

https://quantum-connect.web.cern.ch



© Cristian Schuszter & CERN · QuantumConnect Info Session

Appendix · Numbers to remember

8 × 12
field

6
in a row wins

10
moves cooldown

100 ms
per move

30 s
match limit

1000
max moves

5 MB
max JAR

5–20
bot name chars

2–4
team size

7
games per pair

1
bot per team

© Cristian Schuszter & CERN · QuantumConnect Info Session

Appendix: The QuantumField API

Method	Returns	What it gives you
`getNumberOfRows()`	`int`	8 rows.
`getNumberOfColumns()`	`int`	12 columns.
`isColumnFull(int col)`	`boolean`	Can I drop in this column?
`isSpaceEmpty(int row, int col)`	`boolean`	Is this cell free?
`getField()`	`SpaceOwnership[][]`	A copy of the board.
`belongsToMe(int row, int col)`	`boolean`	Is this disc mine?
`getMoveHistory()`	`List<Move>`	Every move played so far.

Rows: 0 = top, columns: 0 = left. getField() is a copy - simulate on it.

Ownership: EMPTY, FIRST_PLAYER, SECOND_PLAYER. belongsToMe() is relative to you.

© Cristian Schuszter & CERN · QuantumConnect Info Session